Implement an 8-puzzle problem using
i. DFS

```
# 8-Puzzle Problem using Depth-First Search (DFS)

# The goal state representation (0 denotes the blank tile)
GOAL_STATE = (
    1, 2, 3,
    4, 5, 6,
    7, 8, 0
)

# Valid moves: (row_delta, col_delta, Move_Name)
MOVES = [
    (-1, 0, 'Up'),
    (1, 0, 'Down'),
    (0, -1, 'Left'),
    (0, 1, 'Right')
]

def print_board(state):
    """Utility function to print the 3x3 board."""
    for i in range(0, 9, 3):
        print(f" {state[i]} {state[i+1]} {state[i+2]} ")
    print()

def get_neighbors(state):
    """Generates all reachable neighboring states from the current state."""
```

```
    neighbors = []
    # Find the index of the blank tile (0)
    zero_idx = state.index(0)
    row, col = divmod(zero_idx, 3)

    for dr, dc, move_name in MOVES:
        new_row, new_col = row + dr, col + dc

        # Check grid boundaries
        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_idx = new_row * 3 + new_col
            # Swap the blank tile with the adjacent tile
            state_list = list(state)
            state_list[zero_idx], state_list[new_idx] = state_list[new_idx], state_list[zero_idx]
            neighbors.append((tuple(state_list), move_name))

    return neighbors

def dfs_depth_limited(start_state, max_depth=20):
    """
    Executes Depth-First Search with a depth limit and loop avoidance.
    Stack holds: (current_state, path_of_moves, current_depth)
    """
    # LIFO stack for DFS
    stack = [(start_state, [], 0)]

    # Track the minimum depth at which each state was visited
    visited = {start_state: 0}
```

```
    nodes_explored = 0

    while stack:
        current_state, path, depth = stack.pop()
        nodes_explored += 1

        # Check if the goal state is reached
        if current_state == GOAL_STATE:
            return path, nodes_explored

        # If depth limit reached, do not expand further along this branch
        if depth >= max_depth:
            continue

        # Generate neighbor states
        for next_state, move in get_neighbors(current_state):
            # Visit if state not seen, or reached via a strictly shorter path
            if next_state not in visited or depth + 1 < visited[next_state]:
                visited[next_state] = depth + 1
                stack.append((next_state, path + [move], depth + 1))

    return None, nodes_explored


# --- Execution Example ---
if __name__ == "__main__":
    # Example initial state (requires a few steps to solve)
```

```
# Target:
# 1 2 3
# 4 5 6
# 7 8 0
initial_state = (
    1, 2, 3,
    4, 0, 6,
    7, 5, 8
)

print("Initial State:")
print_board(initial_state)

print("Goal State:")
print_board(GOAL_STATE)

depth_limit = 15
print(f"Running DFS (Max Depth Limit: {depth_limit})...\n")
solution_path, nodes = dfs_depth_limited(initial_state, max_depth=depth_limit)

if solution_path is not None:
    print(f"Goal Reached in {len(solution_path)} steps!")
    print(f"Steps: {' -> '.join(solution_path)}")
    print(f"Total nodes explored: {nodes}")
else:
    print(f"No solution found within depth limit of {depth_limit}.")
```

```
Initial State:
 1 2 3
 4 0 6
 7 5 8

Goal State:
 1 2 3
 4 5 6
 7 8 0

Running DFS (Max Depth Limit: 15)...

Goal Reached in 10 steps!
Steps: Right -> Down -> Left -> Up -> Right -> Down -> Left -> Up -> Right -> Down
Total nodes explored: 726
```

]:

8-Puzzle Problem using Iterative Deepening Search (IDS)

```
GOAL_STATE = (
    1, 2, 3,
    4, 5, 6,
    7, 8, 0
)

# Move definitions: (row_delta, col_delta, Move_Name)
MOVES = [
    (-1, 0, 'Up'),
    (1, 0, 'Down'),
```

```
    (0, -1, 'Left'),
    (0, 1, 'Right')
]

def print_board(state):
    """Utility function to print the 3x3 board."""
    for i in range(0, 9, 3):
        print(f" {state[i]} {state[i+1]} {state[i+2]} ")
    print()

def get_neighbors(state):
    """Generates all reachable neighboring states from the current state."""
    neighbors = []
    zero_idx = state.index(0)
    row, col = divmod(zero_idx, 3)

    for dr, dc, move_name in MOVES:
        new_row, new_col = row + dr, col + dc
        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_idx = new_row * 3 + new_col
            state_list = list(state)
            # Swap blank tile with target adjacent tile
            state_list[zero_idx], state_list[new_idx] = state_list[new_idx], state_list[zero_idx]
            neighbors.append((tuple(state_list), move_name))

    return neighbors

def depth_limited_search(state, goal, depth_limit, path, current_branch_visited):
```

```
"""
Recursive Depth-Limited DFS (DLS).
current_branch_visited: set used to prevent cycle loops within the active branch.
"""
if state == goal:
  return path

if depth_limit == 0:
  return None

for next_state, move in get_neighbors(state):
  # Cycle checking along the current exploration path
  if next_state not in current_branch_visited:
    current_branch_visited.add(next_state)

    result = depth_limited_search(
      next_state,
      goal,
      depth_limit - 1,
      path + [move],
      current_branch_visited
    )

    if result is not None:
      return result

    # Backtrack
    current_branch_visited.remove(next_state)
```

```
    return None

def iterative_deepening_search(start_state, goal_state=GOAL_STATE, max_limit=30):
    """
    Iterative Deepening Search (IDS):
    Gradually increases the depth cutoff limit from 0 up to max_limit.
    """
    for depth in range(max_limit + 1):
        print(f"Searching at depth limit: {depth}...")
        visited_in_path = {start_state}

        result = depth_limited_search(
            start_state,
            goal_state,
            depth_limit=depth,
            path=[],
            current_branch_visited=visited_in_path
        )

        if result is not None:
            return result, depth

    return None, None


# --- Execution Example ---
if __name__ == "__main__":
```

```
# Example starting state (solvable in 5 moves)
initial_state = (
    1, 2, 3,
    4, 0, 6,
    7, 5, 8
)

print("Initial State:")
print_board(initial_state)

print("Goal State:")
print_board(GOAL_STATE)

solution, optimal_depth = iterative_deepening_search(initial_state, GOAL_STATE, max_limit=20)

print("-" * 40)
if solution is not None:
    print(f"Optimal solution found at depth {optimal_depth} ({len(solution)} moves):")
    print(f"Path: {' -> '.join(solution)}")
else:
    print("No solution found within the specified depth cutoff limit.")
```

```
Initial State:
 1 2 3
 4 0 6
 7 5 8

Goal State:
 1 2 3
 4 5 6
 7 8 0

Searching at depth limit: 0...
Searching at depth limit: 1...
Searching at depth limit: 2...
----------------------------------------
Optimal solution found at depth 2 (2 moves):
Path: Down -> Right
```

|: